Lecture 14 — Dynamic Programming I: Memoization, Tabulation, 1D DP

> Year Curriculum · Prerequisites: Week 3 (complete search), Week 8 (recursion), Week 11 (modular arithmetic for counting) · Estimated time: 2 × 90 min

1. Overview

Dynamic programming (DP) is the most important technique in competitive programming, and the one students find hardest at first. The idea itself is simple: **a complete search that never solves the same subproblem twice.** In Week 8 we explored recursion trees and saw that some, like naive Fibonacci, contain the same subtree many times. If the answer to a subproblem depends only on a few parameters (the *state*), we can store it the first time and look it up afterwards. An exponential search collapses into a polynomial one.

This lecture builds the habits that make DP manageable:

- recognising when DP applies (overlapping subproblems, optimal substructure);
- a fixed recipe — state, transition, base case, order, answer — that you write down before coding;
- two implementation styles, memoization (top-down) and tabulation (bottom-up);
- counting DPs (number of ways) versus optimisation DPs (min/max);
- extending the state when the transition needs more information;
- reconstructing an optimal solution and reducing memory.

All examples are one-dimensional or nearly so. Week 15 continues with two-dimensional classics (knapsack, LIS, LCS, edit distance, grid paths), and later lectures cover DP on trees, DAGs, bitmasks, intervals and digits.

2. From recursion to DP

2.1 Overlapping subproblems

```
long long fib(int n) { return n <= 1 ? n : fib(n - 1) + fib(n - 2); }
```

The call tree for `fib(n)` has 2F(n+1) − 1 nodes: computing `fib(40)` makes 331,160,281 calls, although there are only 41 distinct arguments. Each distinct subproblem is solved an exponential number of times. That is the signature of a DP opportunity.

2.2 Memoization

Keep the recursion, but cache results:

```
vector<long long> memo(N + 1, -1);
long long fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}
```

Each state is now computed once, after which every further call returns in O(1). Total work is O(n). The sentinel −1 must be a value that can never be a real answer; if every value is possible, keep a separate `vector<bool> done`.

2.3 Tabulation

Alternatively, compute the table directly, in an order where everything a state needs is already known:

```
f[0] = 0; f[1] = 1;
for (int i = 2; i <= n; i++) f[i] = f[i - 1] + f[i - 2];
```

Both versions compute the same table entries using the same formula. They differ only in *who decides the order*: in memoization the recursion discovers it; in tabulation you choose it.

2.4 Optimal substructure

DP also needs the answer to a state to be expressible through answers to smaller states. For optimisation problems this means an optimal solution is made of optimal solutions to subproblems: the cheapest way to reach stone i must end with the cheapest way to reach the stone you jumped from — otherwise you could swap in a cheaper prefix. When a problem lacks this property (for example "longest simple path" in a general graph), DP over a small state does not work.

3. The DP recipe

Before writing any code, write down five things:

1. **State.** A precise one-sentence meaning: "dp[i] = minimum cost to reach stone i". Vague states are the root of most DP bugs.
2. **Transition.** How dp of a state is computed from smaller states. The most reliable way to find it is to ask: **what was the last decision?** (the last jump, the last die, the last coin).

3. **Base case(s).** The smallest states, whose values you know directly.
4. **Order.** For tabulation, an order in which each state's dependencies are computed first (often simply increasing i).
5. **Answer.** Which state, or which combination of states, is the final answer.

The running time is (number of states) × (time per transition), and the memory is the number of states (unless you roll the array, section 6).

4. Worked examples

4.1 Frog 1 (AtCoder DP Contest A)

Stones 0..n−1 have heights h_i. A frog on stone i can jump to i + 1 or i + 2, paying $|h_i - h_j|$. Find the minimum total cost from stone 0 to stone n − 1.

- State: dp[i] = minimum cost to reach stone i.
- Transition: the last jump came from i − 1 or i − 2, so dp[i] = min(dp[i−1] + $|h_i - h_{i-1}|$, dp[i−2] + $|h_i - h_{i-2}|$), the second term only if i ≥ 2.
- Base: dp[0] = 0. Order: increasing i. Answer: dp[n−1].

For h = 10 30 40 20: dp = 0, 20, 30, 30. The answer 30 corresponds to 0 → 1 → 3 (costs 20 and 10). Complexity O(n).

Why not greedy? On h = 30 10 60 10 60 50, "always take the cheaper next jump" pays 20 + 0 + 40 = 60, but the optimal route 0 → 2 → 4 → 5 pays 30 + 0 + 10 = 40. DP examines every route implicitly, but shares the work: there are exponentially many routes and only n states.

4.2 Dice Combinations (CSES 1633) — counting

Count the ordered sequences of die throws (values 1..6) with sum n, modulo $10^9 + 7$.

- State: dp[s] = number of sequences with sum s.
- Transition: classify sequences by their last throw d. Removing it leaves a sequence with sum s − d, and different d give disjoint sets of sequences, so dp[s] = Σ_{d=1}^{min(6,s)} dp[s − d].
- Base: dp[0] = 1 — exactly one sequence (the empty one) has sum 0. Setting dp[0] = 0 would make every count 0.
- Answer: dp[n].

Values: dp = 1, 1, 2, 4, 8, 16, 32, 63, … For n = 3 the four sequences are (1,1,1), (1,2), (2,1), (3). Complexity O(6n).

The two rules of counting DP are visible here: **the cases you sum over must be disjoint** (each object counted once), and **together they must cover every object** (each object counted at least once). Classifying by the last decision usually guarantees both.

4.3 Minimizing Coins (CSES 1634) — optimisation and reconstruction

Coins with values $c_1..c_n$ are available in unlimited supply. Find the fewest coins with total x, or -1.

- State: dp[s] = minimum number of coins summing to s (∞ if impossible).
- Transition: the last coin is some c ≤ s, so dp[s] = 1 + min_c dp[s − c].
- Base: dp[0] = 0. Answer: dp[x], or -1 if it is still ∞.

Trace with coins {1, 5, 7} and x = 11:

s	0	1	2	3	4	5	6	7	8	9	10	11
dp	0	1	2	3	4	1	2	1	2	3	2	3

dp[11] = 1 + min(dp[10], dp[6], dp[4]) = 1 + min(2, 2, 4) = 3, using 5 + 5 + 1. The greedy "largest coin first" would use 7 + 1 + 1 + 1 + 1, five coins. With n ≤ 100 and x ≤ 10^6 the DP does 10^8 cheap operations, which fits in the time limit.

Choose ∞ carefully: with `INT_MAX`, the expression `dp[s - c] + 1` overflows to a negative number and becomes the "minimum". Use something like 10^9 for `int`, or skip states equal to ∞.

Reconstruction. To output the coins themselves, record the decision that achieved each optimum (`last[s] = c`), then walk back from x: print `last[s]`, set s −= `last[s]`, until s = 0. Alternatively, recompute: at state s, any coin c with dp[s − c] + 1 = dp[s] is a valid last coin. The same two options apply to every optimisation DP.

4.4 The order of loops can change what you count

Compare two CSES problems with coins {2, 3, 5} and x = 9:

- **Coin Combinations I (1635)** counts *ordered* sequences: 2+2+5 and 5+2+2 are different. This is exactly Dice Combinations with arbitrary faces: loop over sums outside, coins inside. Answer: 8.
- **Coin Combinations II (1636)** counts *multisets*: 2+2+5 and 5+2+2 are the same. Loop over coins outside and sums inside. When coin c is processed, dp[s] counts multisets that use only the first coins; adding coin c extends them with any number of copies of c. Each multiset is built in coin order, exactly once. Answer: 3 (2+2+5, 3+3+3, 2+2+2+3).

Swapping two loops silently changes the problem. This is the bridge to knapsack DP in Week 15, where the state becomes "using the first i items, total s".

4.5 Removing Digits (CSES 1637)

From n, a step subtracts one of n's (non-zero) decimal digits. Minimum steps to reach 0? State dp[v] = minimum steps from v; transition dp[v] = 1 + min over non-zero digits d of v of dp[v − d]; base dp[0] = 0. Note that the states depend on smaller values only, so increasing v is a valid order even though the transition is "backwards" from the story's point of view. For n = 27 the answer is 5 (27 → 20 → 18 → 10 → 9 → 0).

5. Extending the state: Vacation (AtCoder DP Contest C)

Each day i you choose one of three activities with happiness a[i][0..2], and you may not choose the same activity on two consecutive days. Maximise total happiness.

A state dp[i] = "best total for the first i days" is not enough: to know whether activity j is allowed today, you need yesterday's activity. **When the transition needs information, add it to the state.** Define dp[i][j] = best total for days 0..i, given that day i's activity is j. Then

dp[i][j] = a[i][j] + max_{k ≠ j} dp[i−1][k], with dp[0][j] = a[0][j], answer max_j dp[n−1][j].

With rows (10, 40, 70), (20, 50, 80), (30, 60, 90): dp[0] = (10, 40, 70); dp[1] = (20 + 70, 50 + 70, 80 + 40) = (90, 120, 120); dp[2] = (30 + 120, 60 + 120, 90 + 120) = (150, 180, 210). Answer 210. States: 3n; transitions: 3 each. O(9n).

This "remember the last choice" dimension appears constantly: Codeforces 698A *Vacations* (rest / contest / gym), stock-trading DPs (holding or not), and string DPs that remember the last character.

6. Implementation choices

6.1 Top-down vs bottom-up

	Memoization	Tabulation
Ease of writing	mirrors the recurrence directly	you must choose a valid order
States evaluated	only those reachable from the start	all states
Overhead	function calls, cache checks	tight loops, cache-friendly
Stack depth	equals the longest dependency chain	none
Memory optimisation	difficult	easy (rolling arrays)

Memoization is ideal when the state space is irregular or sparse, or when you are still figuring out the recurrence. Tabulation is faster and safer for large n. A memoized solution with dependency chains of length 10^6 may overflow the stack on some judges.

6.2 Rolling arrays

If dp[i] depends only on dp[i−1] (and perhaps dp[i−2]), keep only the last row(s). Vacation then needs three numbers instead of 3n. Fibonacci needs two variables. The cost: you lose the information needed to reconstruct a solution, so do not roll if you must output the choices.

6.3 Push vs pull

The transitions above are written as *pull*: dp[i] gathers values from its predecessors. The *push* style iterates over states in order and sends each value forward: `dp[i+1] = min(dp[i+1], dp[i] + cost)`. Both compute the same table. Push is convenient when it is easier to list "where I can go from here" than "where I could have come from", and it is common in probability DPs.

7. Reference implementations

7.1 Frog 1 — memoization

```cpp
#include <bits/stdc++.h>
using namespace std;
// AtCoder DP Contest A - Frog 1 (top-down / memoization)
int n;
vector<int> h;
vector<long long> memo;     // -1 = not computed yet
long long solve(int i) {    // min cost to reach stone i from stone 0
    if (i == 0) return 0;
    long long &res = memo[i];
    if (res != -1) return res;
    res = solve(i - 1) + abs(h[i] - h[i - 1]);
    if (i >= 2) res = min(res, solve(i - 2) + abs(h[i] - h[i - 2]));
    return res;
}
int main() {
    cin >> n;
    h.resize(n);
    for (auto &x : h) cin >> x;
    memo.assign(n, -1);
    cout << solve(n - 1) << "\n";
}
```

Recursion depth is up to n = 10^5, which is fine on AtCoder.

7.2 Frog 1 — tabulation

```cpp
#include <bits/stdc++.h>
using namespace std;
// AtCoder DP Contest A - Frog 1 (bottom-up / tabulation)
```

```cpp
int main() {
    int n; cin >> n;
    vector<long long> h(n);
    for (auto &x : h) cin >> x;
    const long long INF = LLONG_MAX / 4;
    vector<long long> dp(n, INF);
    dp[0] = 0;
    for (int i = 1; i < n; i++) {
        dp[i] = dp[i - 1] + llabs(h[i] - h[i - 1]);
        if (i >= 2) dp[i] = min(dp[i], dp[i - 2] + llabs(h[i] - h[i - 2]));
    }
    cout << dp[n - 1] << "\n";
}
```

7.3 CSES 1633 — Dice Combinations

```cpp
#include <bits/stdc++.h>
using namespace std;
// CSES 1633 - Dice Combinations: number of ordered ways to write n as a sum
// of dice throws (1..6), modulo 1e9+7.
int main() {
    const int MOD = 1'000'000'007;
    int n; cin >> n;
    vector<int> dp(n + 1, 0);
    dp[0] = 1;                                      // the empty sequence
    for (int s = 1; s <= n; s++)
        for (int d = 1; d <= 6 && d <= s; d++)
            dp[s] = (dp[s] + dp[s - d]) % MOD;
    cout << dp[n] << "\n";
}
```

7.4 CSES 1634 — Minimizing Coins (with reconstruction)

```cpp
#include <bits/stdc++.h>
using namespace std;
// CSES 1634 - Minimizing Coins: fewest coins summing to x (unlimited supply), or -1.
// Also reconstructs one optimal multiset of coins.
int main() {
    int n, x; cin >> n >> x;
    vector<int> c(n);
    for (auto &v : c) cin >> v;
    const int INF = 1e9;
    vector<int> dp(x + 1, INF), last(x + 1, -1);
    dp[0] = 0;
    for (int s = 1; s <= x; s++)
        for (int v : c)
            if (v <= s && dp[s - v] + 1 < dp[s]) {
                dp[s] = dp[s - v] + 1;
                last[s] = v;                        // remember the choice
            }
    if (dp[x] >= INF) { cout << -1 << "\n"; return 0; }
```

```
    cout << dp[x] << "\n";
    cerr << "coins used:";
    for (int s = x; s > 0; s -= last[s]) cerr << ' ' << last[s];
    cerr << "\n";
}
```

Note `dp[s - v] + 1` cannot overflow because INF = 10^9 leaves room below `INT_MAX`. The reconstruction goes to `cerr` so that the program's standard output matches the judge format.

7.5 Coin Combinations I and II side by side

```
#include <bits/stdc++.h>
using namespace std;
// CSES 1635 (ordered ways) vs CSES 1636 (unordered ways) for coins c and sum x.
int main() {
    const int MOD = 1'000'000'007;
    int n, x; cin >> n >> x;
    vector<int> c(n);
    for (auto &v : c) cin >> v;

    // 1635: sequences (order matters). Sum outer, coin inner.
    vector<int> ordered(x + 1, 0);
    ordered[0] = 1;
    for (int s = 1; s <= x; s++)
        for (int v : c)
            if (v <= s) ordered[s] = (ordered[s] + ordered[s - v]) % MOD;

    // 1636: multisets (order irrelevant). Coin outer, sum inner.
    vector<int> unordered_(x + 1, 0);
    unordered_[0] = 1;
    for (int v : c)
        for (int s = v; s <= x; s++)
            unordered_[s] = (unordered_[s] + unordered_[s - v]) % MOD;

    cout << ordered[x] << " " << unordered_[x] << "\n";
}
```

Input `3 9 / 2 3 5` prints `8 3`.

7.6 Vacation — extra state dimension with a rolling array

```
#include <bits/stdc++.h>
using namespace std;
// AtCoder DP Contest C - Vacation: each day pick activity 0/1/2 with happiness
// a[i][j], never the same activity two days in a row; maximise total.
int main() {
    int n; cin >> n;
    vector<array<long long, 3>> a(n);
    for (auto &r : a) cin >> r[0] >> r[1] >> r[2];
    array<long long, 3> dp = a[0];              // dp[j] = best total ending with j today
```

```
    for (int i = 1; i < n; i++) {
        array<long long, 3> nd{};
        for (int j = 0; j < 3; j++) {
            long long bestPrev = 0;
            for (int k = 0; k < 3; k++) if (k != j) bestPrev = max(bestPrev, dp[k]);
            nd[j] = bestPrev + a[i][j];
        }
        dp = nd;                                    // rolling array: O(1) memory
    }
    cout << *max_element(dp.begin(), dp.end()) << "\n";
}
```

(`bestPrev` may start at 0 because happiness values are non-negative in this problem; in general start from $-\infty$.)

7.7 CSES 1637 — Removing Digits

```
#include <bits/stdc++.h>
using namespace std;
// CSES 1637 - Removing Digits: each step subtract one of the digits of n;
// minimum number of steps to reach 0.
int main() {
    int n; cin >> n;
    vector<int> dp(n + 1, INT_MAX);
    dp[0] = 0;
    for (int v = 1; v <= n; v++)
        for (int t = v; t > 0; t /= 10) {
            int d = t % 10;
            if (d > 0) dp[v] = min(dp[v], dp[v - d] + 1);
        }
    cout << dp[n] << "\n";
}
```

Here `INT_MAX` is safe because every $v \geq 1$ has a non-zero digit, so `dp[v - d]` is always finite by the time it is read.

8. Common mistakes

- **Fuzzy state definition.** If you cannot state what dp[i] means in one sentence, the transitions will be inconsistent. Write it as a comment above the array.
- **State missing information.** If the transition needs to know something (the last activity, whether you hold a stock), it must be part of the state.
- **Wrong base case.** Counting DPs normally have dp[empty] = 1. Optimisation DPs need impossible states set to $\pm\infty$, not 0.
- **Infinity overflow.** `INT_MAX + 1` is negative. Use 10^9 / 10^{18} sentinels or guard the addition.
- **Bad memo sentinel.** If -1 (or 0) can be a real answer, memoization recomputes or returns wrong values.

- **Wrong evaluation order** in tabulation: reading dp[j] before it is computed gives the initial value silently.
- **Loop order in counting.** Ordered vs unordered counts differ only in loop nesting (section 4.4).
- **Missing modulus** on one of several additions, or using `int` for a sum of two values near $10^9 + 7$ without reducing.
- **Array sizes off by one**: states 0..x need x + 1 entries.
- **Stack overflow** with memoization over 10^6+ chained states.

9. Practice problems

Warm-up:

1. **AtCoder DP Contest A — Frog 1** — https://atcoder.jp/contests/dp/tasks/dp_a
2. **AtCoder DP Contest B — Frog 2** — https://atcoder.jp/contests/dp/tasks/dp_b (up to K steps back: O(nK)).
3. **CSES 1633 — Dice Combinations** — https://cses.fi/problemset/task/1633
4. **LeetCode 198 — House Robber** — https://leetcode.com/problems/house-robber/

Core:

5. **AtCoder DP Contest C — Vacation** — https://atcoder.jp/contests/dp/tasks/dp_c
6. **CSES 1634 — Minimizing Coins** — https://cses.fi/problemset/task/1634
7. **CSES 1635 — Coin Combinations I** — https://cses.fi/problemset/task/1635
8. **CSES 1636 — Coin Combinations II** — https://cses.fi/problemset/task/1636
9. **CSES 1637 — Removing Digits** — https://cses.fi/problemset/task/1637
10. **Codeforces 189A — Cut Ribbon** — https://codeforces.com/problemset/problem/189/A (impossible states = $-\infty$).

Challenge:

11. **Codeforces 455A — Boredom** — https://codeforces.com/problemset/problem/455/A (DP over values, not positions).
12. **Codeforces 698A — Vacations** — https://codeforces.com/problemset/problem/698/A (last-activity state).
13. **Codeforces 166E — Tetrahedron** — https://codeforces.com/problemset/problem/166/E (counting with a two-value rolling state).

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 7 "Dynamic programming" (coin problem, counting solutions, constructing a solution): https://cses.fi/book/book.pdf

- AtCoder Educational DP Contest (26 problems, A–Z): https://atcoder.jp/contests/dp — do A–D now, the rest across Weeks 15, 26 and 27.
- USACO Guide, Gold "Introduction to DP": https://usaco.guide/gold/intro-dp
- Codeforces blog, "Dynamic Programming — From Novice to Advanced" (TopCoder tutorial by Dumitru, mirrored widely).
- Errichto's DP lectures on YouTube (series "Dynamic Programming lecture #1–#3").
- Next week (Week 15): knapsack, LIS, LCS, edit distance and grid DP; later Week 20 (tree DP), Week 21 (DP on DAGs), Week 26 (bitmask DP), Week 27 (interval and digit DP).